

CSCE 701 CYBERSECURITY · MODULE 04 · LECTURE NOTES

Network Security

What the network boundary buys, and what it does not.

MODULE OUTCOMES

- Explain what TLS provides and what it does not.
- Explain certificate validation and its failure modes.
- Explain segmentation and what it contains.
- Explain why the perimeter model failed and what replaced it.
- Design network controls proportionate to a threat model.

1 Transport security

TLS GIVES YOU CONFIDENTIALITY, INTEGRITY, AND SERVER IDENTITY — AND NOTHING ELSE

It protects data *in transit* between two endpoints — against eavesdropping and against modification by anyone on the network path. That is a real and valuable guarantee.

It authenticates the *server* you connected to, by means of a certificate — and **it says nothing whatsoever about whether that server is trustworthy, correctly configured, patched, or not currently compromised**. A padlock means you are talking privately to *someone* who holds a certificate for that name.

It does not protect data at rest, data inside the application, or data after it leaves the endpoint — which is where the overwhelming majority of breaches actually occur (Module 12's incident data).

And it does not hide *who* you talked to, when, or how much. The destination address, the timing, and the volume are all visible to anyone on the path — which is sufficient to infer a great deal, and is why traffic analysis is a research field rather than a footnote. **Encrypted DNS and encrypted client hello narrow this and do not close it.**

A CORRECT VALIDATION CHECKS ALL OF THESE:

- the signature chains to a trusted root
- the certificate is within its validity window
- the HOSTNAME matches the subject or a SAN entry
- the certificate is not revoked (OCSP, CRL, or stapled)
- the key usage extension permits this use

THE HISTORIC FAILURES, EVERY ONE OF WHICH SHIPPED IN REAL SOFTWARE:

- the chain was validated and the HOSTNAME WAS NOT CHECKED <- the classic validation disabled "temporarily" during development and left disabled in the release
- a custom trust manager written to accept everything, to get past a self-signed certificate once revocation not checked at all, because checking is slow and the responder is frequently unavailable

THE HOSTNAME ONE IS WORTH DWELLING ON: without it, a valid certificate for ANY domain authenticates ANY server. The chain check passes, the library reports success, and the connection is completely insecure against anyone who can obtain a certificate for any name at all.

SO: USE THE PLATFORM'S VALIDATION. Do not write a trust manager. And if you must pin a certificate, pin to a CA or an intermediate rather than to a leaf, and have a rotation plan -- certificate pinning has caused considerably more outages than it has prevented attacks.

2 Segmentation

SEGMENTATION LIMITS LATERAL MOVEMENT, WHICH IS ITS ONLY JOB

Once an attacker controls one host, the operative question is what else they can reach from it — and a flat network means everything, which is how a single phished workstation becomes a domain compromise.

So segment by trust level and by function — user workstations, application servers, databases, build infrastructure, management interfaces — and default-deny between segments, which turns one compromised workstation into one compromised workstation.

And it is a consequence control rather than a probability control — Module 01 §3's least privilege, applied to the network rather than to a process.

Which means it earns its keep precisely when prevention has already failed. So it is the clearest instance of assume-breach design, and it is the control most often deferred, because its benefit is entirely invisible until there is an incident — which is a budgeting problem rather than a technical one and is worth naming as such.

Control	What it contains	What it does not
Segmentation	Lateral movement between zones.	Movement within a zone — so zone granularity is the design decision.
Egress filtering	Data exfiltration and command-and-control callbacks.	Anything carried over a channel you permit — and HTTPS to a permitted cloud provider is a channel most networks permit.
Firewall or security group	Unexpected inbound connections to unexpected ports.	An attack delivered over an expected port to an expected service, which is how almost all of them arrive.
VPN	Transport-level exposure of internal services to the public internet.	Anything at all, once a client is connected — a VPN grants network position, which §3 argues should not be an authorisation input.
Mutual TLS between services	Service impersonation and unauthenticated internal calls.	A compromised service acting with its own legitimate authority, which is why Module 03's authorisation is still needed.

Egress filtering is the most underused control in this table. Most compromises need to call out — to fetch a second stage, to receive commands, to exfiltrate data — and most networks permit arbitrary outbound connections, which makes restricting egress unusually high-value for its cost. An allowlist of outbound destinations is tedious to maintain and it breaks a large class of attack entirely.

3 The perimeter's failure

THE PERIMETER MODEL ASSUMED A BOUNDARY THAT NO LONGER EXISTS

'Inside the network is trusted, outside is not' worked tolerably when everything was in one building, on one network, with one way in.

And then: remote work, cloud infrastructure, contractor and vendor access, mobile devices, personal devices, and software-as-a-service holding your data. There is no inside any more — the boundary is not weakened, it is absent.

So the replacement is to authenticate and authorise every request regardless of where it came from — no implicit trust derived from network position, which is the 'zero trust' idea with the marketing removed.

Which is really Module 03's authorisation applied uniformly. The network stops being an authorisation input, and the principal's identity and the device's posture become the inputs instead — so the architecture change is downstream of getting identity and authorisation right, which is why the sequencing below matters.

- **Strong identity for every principal**, human and service — **which is why Module 02 and Module 07 come before this one**, and why an organisation without working identity management cannot do any of this.
- **Authorisation on every request**, at a chokepoint (Module 03 §3), **with the network position as at most one signal among several** rather than as the decision.
- **Device posture as a signal** where you control the devices — patch level, disk encryption, management enrolment — **and honestly where you do not**, since a contractor's laptop cannot be assessed.
- **And segmentation anyway. Removing implicit network trust does not remove the value of containing a compromise** — the two are complementary rather than alternatives, and treating the new model as a reason to flatten the network is a misreading.
- **Which makes it an expensive, multi-year programme**, and **the honest sequencing is identity first, then authorisation, then the network**. 'Zero trust' is not a product — it is the consequence of getting identity and authorisation right, **and a vendor selling it as a network appliance has inverted the dependency**.

4 Proportionality

NETWORK CONTROLS ARE FREQUENTLY THE WRONG PLACE TO SPEND

Published incident data consistently shows stolen or guessed credentials, unpatched software, and misconfiguration as the leading initial access vectors — none of which a firewall addresses at all.

And network controls are visible, purchasable, and satisfying to deploy, which makes them attractive relative to the far less visible work of fixing authorisation coverage, maintaining a patch cadence, and reviewing configurations — **which is an incentive problem rather than an engineering one** and is Module 12's subject.

So the proportionate ordering is usually: identity and authentication (Module 02), authorisation (Module 03), patching and dependency hygiene (Module 08), then segmentation and egress filtering, then everything else.

With the important exception that segmentation and egress filtering are cheap and high-value — so they are the network controls to do first, and they are the ones most often skipped in favour of more expensive and considerably less useful ones.

- **Is TLS used everywhere, including for traffic between internal services? Internal plaintext assumes the network is trusted**, which §3 says it is not.
- **Is certificate validation enabled everywhere, including inside every client library and every service-to-service call? §1's failures are all in client code.**
- **Can any host reach any other host? If so, one compromise is a full compromise**, and the fix is cheap relative to its value.
- **Can hosts make arbitrary outbound connections? If so, exfiltration and command-and-control are entirely unimpeded**, and the detection in Module 09 is your only line.
- **And is the network used as a sole authorisation input anywhere? 'Internal only' as a security control is the pattern to find and remove — and this question finds the most severe findings**, because an unauthenticated service protected only by being on the internal network is exactly one compromise away from being public.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Anderson — Security Engineering, the networking chapters (free PDF)	§1 and §3 in context , with the history of why the perimeter model was adopted and why it failed.
Georgiev et al. — The Most Dangerous Code in the World (free)	§1's certificate validation failures, surveyed across real software — and the title is not hyperbole. Required reading.
NIST SP 800-207 — Zero Trust Architecture (free)	§3 without the marketing — the architectural components and the sequencing.

Resource	What it covers for this module
Verizon Data Breach Investigations Report (free, annual)	§4's incident data — read the initial-access section each year, and compare it against where your budget goes.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Capture your own TLS traffic** and report what an observer on the path can still see.
- 2 **Estimate what the destinations and volumes alone reveal** about a browsing session.
- 3 **Write a client that validates the chain and not the hostname**, and demonstrate the consequence against your own test server.
- 4 **Then fix it** by using the platform's validation.
- 5 **Map your network's reachability**: can any host reach any other?
- 6 **Implement egress filtering** on one host and report what broke and what it would have prevented.
- 7 **Find a service in your environment protected only by being internal**, and add authentication.
- 8 **Check certificate validation** in every client library your system uses.
- 9 **Read the current breach report's initial-access section** and compare against your own control spending.
- 10 **Write the proportionate ordering** for your own system, with reasons.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does TLS provide, and what does it explicitly not?
- 2 What remains visible to an observer on the path?
- 3 Name five certificate validation checks and the historic failures of each.
- 4 Why is the hostname check the critical one?
- 5 What is the caveat on certificate pinning?
- 6 What does segmentation contain, and what kind of control is it?
- 7 Name five network controls and what each does not contain.
- 8 Why is egress filtering underused?
- 9 Why did the perimeter model fail, and what does the replacement require?
- 10 Give the proportionate ordering and the exception to it.